

Ingeniería del Software 3

Clase 4 · CI: Pipelines as Code

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

Lo que vas a ver en este video

- El práctico de hoy se hace sobre el repositorio de tu app del semestre — el mismo del TP2
 - Y esta vez la app importa: el pipeline la construye con tu propio Dockerfile. Si todavía no está en el repo, ésa es la prioridad de la semana

Bloque	De qué se trata
La teoría	CI como práctica · el archivo · anatomía (workflow/job/step) · triggers · runners · los dos rieles · el cache de capas · dónde va una credencial · las dos reglas de equipo
El gate	El pipeline como requisito de merge — adonde apunta toda la clase
El práctico	Qué tenés que entregar y con qué se evalúa
Y después	El paso a paso completo, en el video del TP4

CI es una práctica, no una herramienta

"Integrar el trabajo de todos, con frecuencia, verificando cada integración automáticamente."

- El problema que ataca (¿te suena de la clase 1?): integrar tarde es integrar caro
- Ramas de semanas → el día de la integración es un evento traumático ('integration hell')
- Integración diaria + verificación automática → los problemas se detectan en MINUTOS, cuando el cambio es chico y está fresco
- **Verificar = que el proyecto SE CONSTRUYA + que los TESTS PASEN. Hoy cableamos lo primero; los tests llegan en el TP5, como una etapa más del mismo pipeline**
- Se puede tener pipeline sin hacer CI (ramas eternas + pipeline decorativo) — y CI sin gran pipeline (disciplina + un script). La práctica es lo que cuenta

El archivo: esto es un pipeline

```
name: CI
on:                                     # ① QUÉ lo dispara
  pull_request:
    branches: [main]                  # ← verifica ANTES del merge
  push:
    branches: [main]
jobs:                                  # ② QUÉ unidades tiene
  build-backend:                       # el NOMBRE del job (acordate)
    runs-on: ubuntu-latest            # ③ DÓNDE corre
    steps:                             # ④ QUÉ hace, en orden
      - name: Qué estamos verificando # ↓ un paso TUYO: run: + un comando
        env:
          RAMA: ${github.head_ref || github.ref_name}
        run: echo "Rama $RAMA · commit $GITHUB_SHA"
      - name: Bajar el código al runner # ↓ pasos de OTROS: uses: + su nombre
        uses: actions/checkout@v6
      - name: Construir la imagen del backend
        uses: docker/build-push-action@v7
        with:
          context: ./backend           # ← tu Dockerfile del TP2
          push: false                  # hoy no se publica en ningún lado
          tags: backend:ci             # su nombre, puertas adentro
```

Un archivo de texto adentro de tu repo (.github/workflows/ci.yml) que NO compila por su cuenta: le pide a tu Dockerfile que construya. Todo lo que sigue sale de acá.

Anatomía: workflow → jobs → steps

workflow: CI (on: push · pull_request)

job: build-backend

step: checkout

step: buildx

step: docker build

step: (push, más
adelante)

job: build-frontend

step: checkout

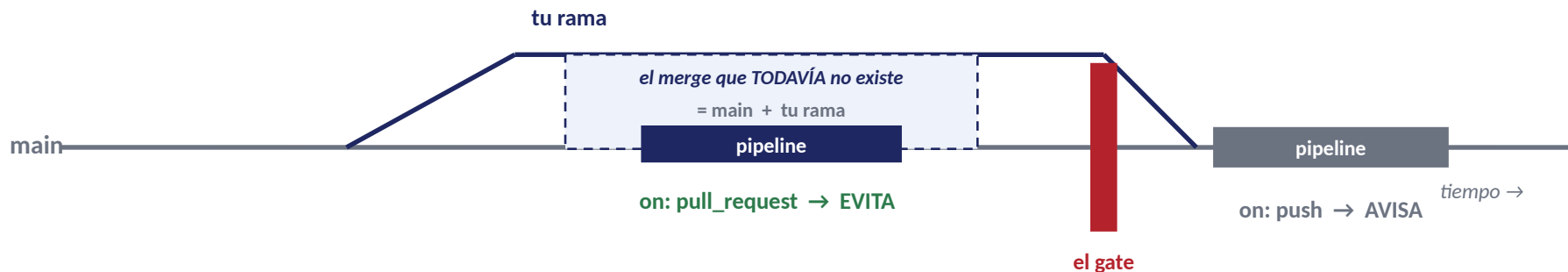
step: buildx

step: docker build

step: (push, más
adelante)

*jobs en PARALELO,
cada uno en un runner
limpio e independiente*

Triggers: cuándo corre el pipeline



Y los otros dos	Cuándo dispara
workflow_dispatch	Disparo manual: un botón para lanzarlo vos
schedule (cron)	Periódico — builds nocturnos, chequeos que corren solos

Runners: dónde corre todo esto

	Hosted (GitHub/Microsoft)	Self-hosted (tu máquina)
Mantenimiento	Cero — te lo dan listo	Tuyo: instalación, updates, seguridad
Estado	Efímero: máquina LIMPIA por job	Persistente (cuidado con el estado fantasma)
Costo	Gratis en repos públicos (ilimitado)	Tu hardware, tu luz
Cuándo	El default correcto — casi siempre	Hardware/red propios... y el FALLBACK LOCAL de esta materia (TP6/7)

- **Guarden 'self-hosted runner' en la memoria: es la pieza que permite desplegar A TU PROPIA MÁQUINA desde el pipeline**
-  **Corre en TU equipo lo que diga el workflow: GitHub los desaconseja en repos PÚBLICOS (un PR desde un fork podría ejecutar código ajeno en tu máquina)**
- Si un free tier de deploy muere a mitad de semestre, el plan C de la materia es exactamente ese (TP6/TP7)

Los dos rieles: los mismos conceptos, otros nombres

Concepto	GitHub Actions (riel de la materia)	Azure Pipelines
El proceso completo	workflow (un YAML en .github/workflows/)	pipeline (azure-pipelines.yml)
Qué lo dispara	event (on: push, pull_request...)	trigger
Unidad que corre en una máquina	job	job (agrupados en stages)
Paso individual	step (run: o uses:)	step (script o task)
Bloque reutilizable de terceros	action (Marketplace)	task
La máquina que ejecuta	runner (hosted o self-hosted)	agent (Microsoft-hosted o self-hosted)

El riel con soporte de cátedra y el de los videos es GitHub Actions. Azure es una opción válida: el enunciado trae la tabla completa y las condiciones de ese camino en 2026.

El cache: reutilizar lo que no cambió



RUN dotnet build	se rehace
COPY . .	se rehace
RUN dotnet restore	CACHED
COPY *.csproj	CACHED
FROM .../sdk:8.0	CACHED

Cambiaste código, no dependencias → las tres de abajo se reutilizan.
Por eso el Dockerfile del TP2 copia primero los archivos de proyecto y después el código.

Cache = lo que ya hiciste una vez y no querés repetir. Si no está: más lento, NO roto.

Secrets: dónde va una credencial si tu repo es público

- Un pipeline que despliega o publica necesita credenciales. En el YAML no van: tu repositorio es público
- Se cargan como secretos en la configuración del repo: quedan cifrados y llegan al pipeline recién al correr, como variable de entorno
- **Los logs de un repo público son públicos y permanentes: si se filtra una credencial real, no se borra — se REVOCA**
- Tu pipeline de hoy no necesita ninguna. La primera de verdad llega cuando empiece a desplegar tu app

Las dos reglas culturales de un equipo que hace CI

Regla 1

El build roto es la prioridad número UNO

main en rojo → nadie apila trabajo encima; se arregla o se revierte AHORA. Un rojo tolerado días convierte el pipeline en decoración.

Regla 2

Si no pasó por el pipeline, no existe

Nada llega a main sin que las máquinas lo verifiquen. Es la protección del TP1, ahora con verificación automática además de la humana.

Las dos se hacen cumplir con configuración, no con voluntad — hoy cableamos la segunda (el gate).

El mismo PR, con el gate y sin el gate

MAL · el pipeline informa

PR #7 · agregar el endpoint /health

✗ build-backend — falló

Merge pull request

el botón sigue habilitado

**Alguien mergea igual — y main queda rota
para todo el equipo**

BIEN · el pipeline manda

PR #7 · agregar el endpoint /health

✗ build-backend — falló

Merge pull request

 Required check failing

**No hay botón que apretar hasta el verde:
main no se rompe porque no puede**

*El pipeline es EXACTAMENTE el mismo de los dos lados. Lo único que cambia es una casilla del repo:
que ese check sea obligatorio. Eso es el gate.*

El gate: el pipeline como requisito de merge

La idea: main deja de aceptar un merge si esos checks no están en verde.

Riel	Cómo se llama	Dónde se configura
GitHub Actions	Required status checks	Settings → Branches (o el comando gh api del enunciado)
Azure Pipelines	Build validation	Repos → Branches → Branch policies de la rama destino

jobs: build-backend



el nombre que
main exige

el MISMO nombre, escrito dos veces (en el comando se llama contexts): si le agregás name: al job, el gate espera un check que ya no existe

- strict: true — tu rama tiene que tener incorporado el main actual antes de mergear
 - Te pasa esta semana: con dos PRs abiertos, mergeás uno y el verde del otro queda viejo → botón Update branch
- **Con esto, la regla 2 ('si no pasó por el pipeline, no existe') deja de ser un deseo: es configuración**



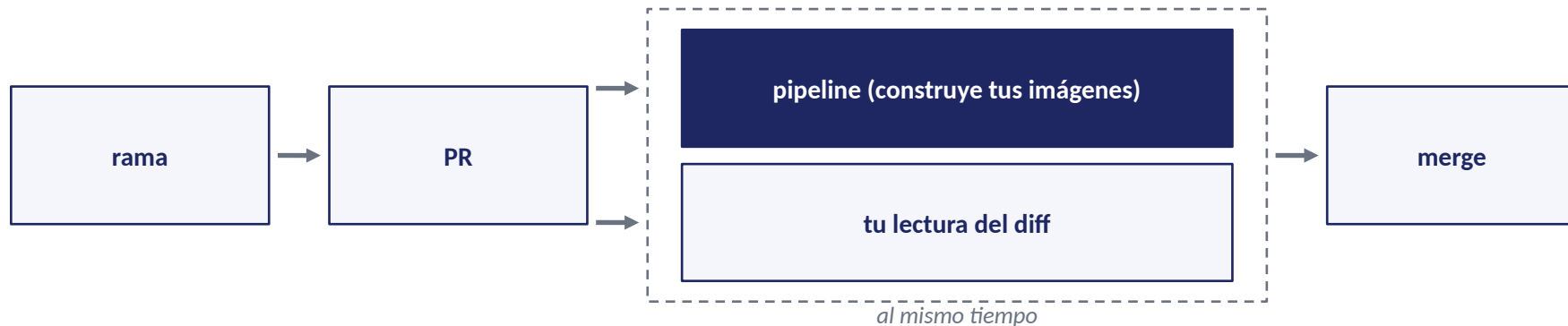
Las DOS, o no entra

Volvamos al archivo: ahora lo lees entero

```
name: CI
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
jobs:
  build-backend:
    runs-on: ubuntu-latest
    steps:
      - name: Qué estamos verificando
        env:
          RAMA: ${github.head_ref || github.ref_name}
        run: echo "Rama $RAMA · commit $GITHUB_SHA"
      - name: Bajar el código al runner
        uses: actions/checkout@v6
      - name: Preparar el constructor que sabe cachear # ← sin esto, el build FALLA
        uses: docker/setup-buildx-action@v4
      - name: Construir la imagen del backend
        uses: docker/build-push-action@v7
        with:
          context: ./backend
          push: false
          tags: backend:ci
          cache-from: type=gha,scope=backend # ← trae las capas de la corrida anterior
          cache-to: type=gha,mode=max,scope=backend # ← las guarda al terminar
```

El mismo archivo del principio, más el constructor y las dos líneas del cache. Ahora sabés qué hace cada bloque y por qué está.

El flujo completo de tu repo (y el badge)



El merge exige las DOS: ninguna espera a la otra. Y al mergear, la tarea del TP3 se cierra sola: los cuatro TP3, funcionando junto.

- **El badge: el semáforo en la puerta del repo**
 - Es una línea de markdown que se pega en el README — la exacta está en la guía. En Azure DevOps el badge existe igual, con otra URL: se copia desde el menú del pipeline (Status badge)
- Parece cosmético; es cultura: el estado del proyecto, visible para cualquiera. Un badge rojo a la vista incomoda lo justo para que alguien lo arregle
- **Y queda en tu repo público: un badge verde con un pipeline real detrás habla por vos ante cualquiera que lo abra**

De cero al gate: el recorrido completo

- **Los cinco pasos de la guía (§3) — el video del TP4 los hace uno por uno:**
 - 1. El workflow: reemplazás el ci.yml del TP3 por checkout + build con tu Dockerfile (entra por PR, como todo)
 - 2. El frontend como segundo job, en paralelo, y el cache de capas (comparar 1ra vs 2da corrida)
 - 3. El gate: required status checks + strict sobre main
 - 4. La verificación central: romper la compilación → PR BLOQUEADO → fix → verde → merge
 - 5. El badge en el README
- **El paso 4 es la evidencia central de tu TP — que la secuencia entera quede visible en el PR**

Pipeline as Code: el pipeline vive en el repo

El pipeline es un archivo del repo...	...y por eso
Se revisa	Cambiar el pipeline es un PR más: con diff y con revisión, igual que el código
Se versiona	El historial dice quién lo tocó y cuándo — y se revierte como cualquier commit
Se reproduce	Clonás el repo y el pipeline viene adentro: no hay nada que reconstruir a mano
Viaja con la rama	Cada rama puede probar su propia versión del pipeline, sin romperle nada a nadie

- **4ta aparición del patrón de la materia — todo lo importante se declara, no se hace a mano:**
 - TP1: protecciones • TP2: entorno (Dockerfile/compose) • TP3: plan enlazado • HOY: el proceso de build • TP8: la infraestructura
 - Dónde vive cada una: entorno, build e infraestructura son archivos DEL repo; protecciones y plan viven en la configuración de la plataforma

TP4 — Entregables

Entregable	Contenido
URL del repo público	Workflow + historial de corridas + PRs
decisiones.md	Estructura del pipeline · qué cachea y qué pasa sin cache · por qué construís con tu Dockerfile · problemas · uso de IA

Cómo se evalúa este práctico: 25% configuración técnica · 25% decisiones.md · 50% defensa oral

No hay evidencias.md: tu repo es público — las corridas, los checks y el badge ya se ven.

Cierre — rumbo a P1

1

TP4 esta semana: CI verde + gate demostrado

La secuencia rojo→bloqueado→fix→verde→merge es tu evidencia central

2

Repasá los 4 enunciados con sus preguntas de defensa

Salen de la teoría vista — decisiones.md es tu mejor material de repaso

3

Ensayá la navegación en vivo

Clone limpio + compose up · push rechazado · historia→commit · gate bloqueando

4

Próxima clase: P1 — defensa de TPs 1-4

Individual, repo abierto. La defensa es la mitad de la nota; la otra mitad ya está en tu repo